

Module 6: Embedded SQL

Spring 2026

Reading Assignments and References

To be read during the Week of June 8–12:[†]

- ▶ Chapter 5, Section 5.1

[†]7th edition of course textbook: Avi Silberschatz, Henry F. Korth, and S. Sudarshan *Database System Concepts*.

Outline

Unit 1: **Static Embedded SQL (in C)**

Unit 2: Stored Procedures

Unit 3: ODBC

Static Embedded SQL

SQL Statements are **embedded** in a **host language** such as C, C++, FORTRAN, and so on.

The application is *preprocessed* to produce a pure host language program with library calls.

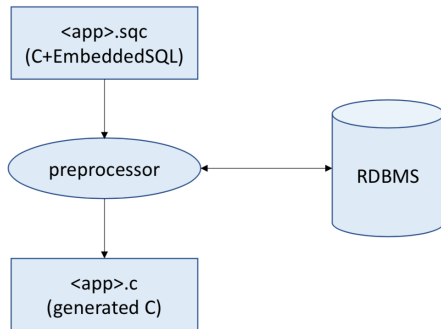
Advantages:

- ▶ Preprocessing of (static) parts of queries becomes possible.
- ▶ Much easier to use.
- ▶ Potentially more secure.

Disadvantages:

- ▶ Requires a pre-compiler.
- ▶ Generated host language program might be bound to a particular RDBMS engine.
⇒ DB2 has this property

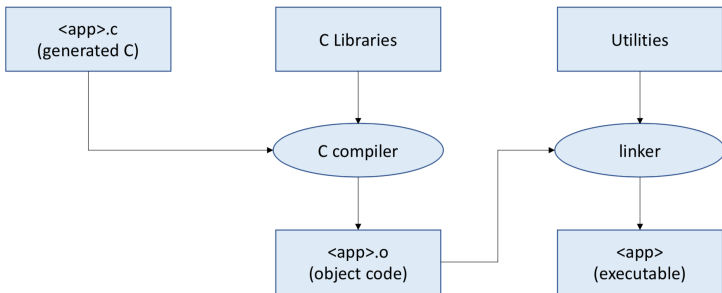
Development Process for Embedded SQL Applications



► **Warning:** Not all RDBMSs communicate with the server to *compile* at this time.

⇒ some products simply generate a CLI interface, e.g., ODBC

Development Process (cont'd)



Reminder: For DB2, <app> will be bound to a particular server engine.

Static Embedded SQL (cont.)

Considerations:

- ▶ How much can SQL be parameterized?
 - ⇒ how to pass parameters into SQL
 - ⇒ how to get results of queries
 - ⇒ exception handling
- ▶ Static versus dynamic SQL statements.

How much does the DBMS know about an application?

⇒ **precompiling:** `PREP`

⇒ **binding:** `BIND`

Application Structure

```
Include SQL support (SQLCA, SQLDA)
```

```
main(int argc, char **argv)
```

```
{
```

```
    Declarations
```

```
    Connect to Database
```

```
    Do your work
```

```
    Process errors
```

```
    Commit/Abort and Disconnect
```

```
};
```

Declarations

- ▶ Include SQL communication area:

```
EXEC SQL INCLUDE SQLCA;
```

Defines:

⇒ return code of SQL statements (sqlcode)

⇒ error messages (if any)

⇒ **required**

- ▶ An SQL statement can be inserted by embedding the following where C statements can occur:

```
EXEC SQL <sql statement>;
```

Host Variables

Host Variables

SQL statements can have parameters that are local or global variables in the embedding language.

- ▶ Host variables communicate *single scalar values* between an SQL a statement and the embedding language.
- ▶ Must be declared within an SQL declare section:

```
EXEC SQL BEGIN DECLARE SECTION;
```

(only declarations of host variables in host language go here)

```
EXEC SQL END DECLARE SECTION;
```
- ▶ Can then be used in `EXEC SQL` statements.
- ▶ To distinguish them from SQL identifiers, they are prefixed by “:”, the colon character.

Exception Handling

What if a SQL statement fails?

- ▶ Check `sqlcode != 0`, or
- ▶ Use exception handling SQL commands:

```
EXEC SQL WHENEVER SQLERROR GO TO <label>;  
EXEC SQL WHENEVER SQLWARNING GO TO <label>;  
EXEC SQL WHENEVER NOT FOUND GO TO <label>;
```

⇒ `<label>` must be in scope

Connecting to a Database (sample1)

```
#include <stdio.h>
#include <stdlib.h>
#include "util.h"

EXEC SQL INCLUDE SQLCA;

int main(int argc, char *argv[]) {

    EXEC SQL BEGIN DECLARE SECTION;
        char db[6] = "cs348";
    EXEC SQL END DECLARE SECTION;

    printf("Sample C program: CONNECT\n" );

    EXEC SQL WHENEVER SQLERROR  GO TO error;

    EXEC SQL CONNECT TO :db;

    printf("Connected to DB2\n");

    EXEC SQL COMMIT;
    EXEC SQL CONNECT reset;
    exit(0);
```

Connecting to a Database (sample1, cont'd)

```
error:
    check_error("My error",&sqlca);
    EXEC SQL WHENEVER SQLERROR CONTINUE;

    EXEC SQL ROLLBACK;
    EXEC SQL CONNECT reset;
    exit(1);
}
```

Adding SQL DML Statements

With a surrounding infrastructure, now consider adding executable DML statements:

- ▶ Simple statements:
 - ⇒ “constant” statements
 - ⇒ statements with parameters
 - ⇒ queries returning a single tuple
- ▶ General queries with many answers.
- ▶ Dynamic queries (ODBC).

Executing Simple Statements (sample2)

Write a program that prints out the title of the publication for each publication identifier supplied as an argument.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "util.h"

EXEC SQL INCLUDE SQLCA;

int main(int argc, char *argv[]) {
    int i;
    char pubidstr[10];

    EXEC SQL BEGIN DECLARE SECTION;
        char db[6] = "cs348";
        int pubid;
        char title[30];
    EXEC SQL END DECLARE SECTION;

    printf("Sample C program: SAMPLE2\n" );

    EXEC SQL CONNECT TO :db;

    printf("Connected to DB2\n");
```

Executing Simple Statements (sample2, cont'd)

```
EXEC SQL WHENEVER SQLERROR GO TO error;

for (i=1; i<argc; i++) {
    strncpy(pubidstr, argv[i], 8);
    pubid = atoi(pubidstr);

    EXEC SQL WHENEVER NOT FOUND GO TO nope;
    EXEC SQL SELECT title INTO :title
        FROM    publication
        WHERE   pubid = :pubid;

    printf("%d: %s\n",pubid,title);
    continue;
nope:
    printf("%d: *** not found *** \n",pubid);
};

EXEC SQL COMMIT;
EXEC SQL CONNECT reset;
exit(0);

error:
    check_error("My error",&sqlca);
    EXEC SQL WHENEVER SQLERROR CONTINUE;
    EXEC SQL ROLLBACK;
    EXEC SQL CONNECT reset;
    exit(1);
}
```

Executing Simple Statements (sample2, cont'd)

```
./sample2 1 7 3 5
```

```
Sample C program: SAMPLE2  
Connected to DB2  
1: Mathematical Logic  
7: *** not found ***  
3: Principles of DB Systems  
5: *** not found ***
```

It is important that at most *one* title is returned for each publication identifier.

NULLs and Indicator Variables

- ▶ What if a result value is NULL?

⇒ not a valid value in the datatype

- ▶ Embedded SQL uses an extra **indicator** variable, e.g.:

```
smallint ind; // (within a BEGIN/END DECLARE SECTION)
...
EXEC SQL SELECT firstname INTO :firstname
                        INDICATOR :ind
                        FROM ... ;
```

If `ind < 0` then `firstname` is NULL.

- ▶ If the indicator variable is not provided and the result is a null we get an **run-time error**.
- ▶ The same rules apply for host variables in updates.

Impedance Mismatch

SQL CURSOR

For SQL queries possibly returning more than one answer, an SQL CURSOR is defined and used in an iterator protocol.

1. Declaring a cursor:

```
EXEC SQL DECLARE <name> CURSOR FOR  
    <query>;
```

2. An iterator protocol using a cursor:

```
EXEC SQL OPEN <name>;  
EXEC SQL WHENEVER NOT FOUND GO TO end;  
for (;;) {  
    <set up host parameters>  
    EXEC SQL FETCH <name> INTO <host variables>;  
    <process the fetched tuple>  
};  
end:  
EXEC SQL CLOSE <name>;
```

Executing Queries with Many Answers (sample3)

Write a program that lists all author names and publication titles with author name matching a pattern given as an argument.

```
main(int argc, char *argv[]) {
    ...
    strncpy(apat,argv[1],8);

    EXEC SQL DECLARE author CURSOR FOR
        SELECT name, title
        FROM author , wrote, publication
        WHERE name LIKE :apat
        AND aid=author
        AND pubid=publication;

    EXEC SQL OPEN author;
    EXEC SQL WHENEVER NOT FOUND GO TO end;
    for (;;) {
        EXEC SQL FETCH author INTO :name, :title;
        printf("%10s -> %10s: %s\n",apat,name,title);
    };
    end:
    EXEC SQL CLOSE author;
    ...
}
```

Executing Queries with Many Answers (sample3, cont'd)

```
./sample3 "%"
```

Sample C program: SAMPLE3

Connected to DB2

```
% ->      Sue: Mathematical Logic
% ->      Sue: Trans. on Databases
% ->      Sue: Query Languages
% ->      John: Trans. on Databases
```

```
./sample3 "%u_"
```

Sample C program: SAMPLE3

Connected to DB2

```
%u_ ->    Sue: Mathematical Logic
%u_ ->    Sue: Trans. on Databases
%u_ ->    Sue: Query Languages
```

Summary

► Declarations:

```
EXEC SQL INCLUDE SQLCA;  
EXEC SQL BEGIN DECLARE SECTION;  
    <host variables here>  
EXEC SQL END DECLARE SECTION;
```

► Simple statements:

```
EXEC SQL <SQL statement>;
```

► Queries (with multiple answers):

```
EXEC SQL DECLARE <id> CURSOR FOR <qry>;  
EXEC SQL OPEN <id>;  
do {  
    EXEC SQL FETCH <id> INTO <vars>;  
} while (SQLCODE == 0);  
EXEC SQL CLOSE <id>;
```

► Don't forget to check errors!

Outline

Unit 1: Embedded SQL (in C)

Unit 2: **Stored Procedures**

Unit 3: ODBC

Stored Procedures

Idea

A *stored procedure* executes application logic directly inside the RBDMS server.

Possible implementations:

- ▶ An ability to invoke externally-compiled application; and/or
- ▶ Support for **SQL/PSM** (or a vendor-specific equivalent).

Reasons for stored procedures:

- ▶ Reduces volume of data transfer between client and server;
- ▶ Centralizes application code at the server; and
- ▶ Conceptual schema enhancement.

A Stored Procedure Example: Atomic-Valued Function

```
CREATE FUNCTION sumSalaries(dept CHAR(3))  
    RETURNS DECIMAL(9,2)  
LANGUAGE SQL  
RETURN  
    SELECT sum(salary)  
    FROM employee  
    WHERE workdept = dept
```

A Stored Procedure Example: Atomic-Valued Function

```
db2 => SELECT deptno, sumSalaries(deptno) AS sal \  
=> FROM department
```

DEPTNO	SAL
A00	128500.00
B01	41250.00
C01	90470.00
D01	-
D11	222100.00
D21	150920.00
E01	40175.00
E11	104990.00
E21	95310.00

9 record(s) selected.

A Stored Procedure Example: Table-Valued Function

```
CREATE FUNCTION deptSalariesF(dept CHAR(3))  
    RETURNS TABLE(salary DECIMAL(9,2))  
    LANGUAGE SQL  
RETURN  
    SELECT salary  
    FROM employee  
    WHERE workdept = dept
```

A Stored Procedure Example: Table-Valued Function

```
db2 => SELECT * FROM TABLE \  
=> (deptSalariesF(CAST('A00' AS CHAR(3)))) AS s
```

SALARY

52750.00

46500.00

29250.00

3 record(s) selected.

A Stored Procedure Example: Branching

```
CREATE PROCEDURE UPDATE_SALARY_IF
    (IN employee_number CHAR(6), INOUT rating SMALLINT)
LANGUAGE SQL
BEGIN
    DECLARE not_found CONDITION FOR SQLSTATE '02000';
    DECLARE EXIT HANDLER FOR not_found
        SET rating = -1;
    IF rating = 1 THEN
        UPDATE employee
        SET salary = salary * 1.10, bonus = 1000
        WHERE empno = employee_number;
    ELSEIF rating = 2 THEN
        UPDATE employee
        SET salary = salary * 1.05, bonus = 500
        WHERE empno = employee_number;
    ELSE
        UPDATE employee
        SET salary = salary * 1.03, bonus = 0
        WHERE empno = employee_number;
    END IF;
END
```

Outline

Unit 1: Embedded SQL (in C)

Unit 2: Stored Procedures

Unit 3: **ODBC**

ODBC: An SQL Call Level Interface (CLI)

- ▶ An interface defined by a C library of function calls defined by Microsoft.
- ▶ For CLI, applications are developed entirely in a host language.
 - ⇒ no pre-compilation so no type checking at compile time
 - ⇒ less transparent compared to static embedded SQL
- ▶ The SQL CLI standard incorporates ODBC and X/Open standards.

Three kinds of *handles* in an ODBC program for:

1. an **environment** (exactly one per application thread);
2. a database engine **connection**; and
3. an SQL **statement**.

Recall that dynamic embedded SQL has statement handles only.

⇒ an executing thread has at most one ongoing transaction

Connect and Disconnect in ODBC

```
int main()
{
    SQLHENV    henv;
    SQLHDBC    hdbc;
    SQLRETURN  rc;
    SQLCHAR    server[SQL_MAX_DSN_LENGTH + 1] = "DBCLASS";
    SQLCHAR    uid[19] = "<your uid>";
    SQLCHAR    pwd[31] = "<your password>";

    SQLAllocEnv(&henv);
    SQLAllocConnect(henv, &hdbc);

    rc = SQLConnect(hdbc, server, SQL_NTS, uid, SQL_NTS, pwd, SQL_NTS);
    if (rc != SQL_SUCCESS) {
        printf("Error connecting to %s\n", server); exit(1);
    } else printf("Connected to %s\n", server);

    /* DO SOMETHING HERE */

    SQLDisconnect(hdbc);
    SQLFreeConnect(hdbc);
    SQLFreeEnv(henv);
}
```

Exception Handling in ODBC (errors)

- ▶ Each `SQLxxx` function returns an error code.
 - ⇒ similar to `libc` functions
 - ⇒ should be checked after each `SQLxxx` call
- ▶ Return codes:
 1. `SQL_SUCCESS`
 2. `SQL_ERROR`
- ▶ The `SQLError` function is used to obtain more details.

SQL Statements in ODBC

- ▶ `SQLAllocStmt` (allocates a statement handle)
- ▶ `SQLExecDirect` (execute)
- ▶ `SQLPrepare` (compile statement)
- ▶ `SQLExecute` (execute compiled statement)
- ▶ `SQLSetParam` (initialize a procedure parameter)
- ▶ `SQLNumResultCols` (number of result columns)
- ▶ `SQLBindCol` (“host variables” in ODBC)
- ▶ `SQLGetData` (obtaining values of result columns)
- ▶ `SQLFetch` (cursor access in ODBC)
- ▶ `SQLError` (obtains diagnostics)
- ▶ `SQLRowCount` (number of affected rows)
- ...
- ▶ `SQLFreeStmt` (frees a statement handle)

Parameters

1. Parameter markers:

⇒ specified by '?' in the text of the query

`SQLNumParams`

`SQLBindParameter`

2. Results of queries:

⇒ specified by the number of resulting columns

`SQLNumResultsCol`

`SQLDescribeCol`

`SQLBindCol` or `SQLGetData`

3. Number of affected tuples (updates):

`SQLRowCount`

Parameter (cont'd)

The parameters are *bound* to buffers by `SQLBindParameter`.

```
SQLBindParameter(stmt-handle,  
                 param-nr,  
                 inp/out,  
                 c-type,  
                 db-type,  
                 db-prec,  
                 db-scale,  
                 val-ptr, val-len,  
                 val-NULL-ptr)
```

Substitutes a value pointed to by `val-ptr` (with length `val-len`, indicator variable `val-NULL-ptr`, and C data type `c-type`) for the `param-nr`-th parameter of `stmt-handle` (using database type `db-type`).

Example

```
SQLCHAR  stmt[] = "UPDATE author SET url = ? WHERE aid = ?";

SQLINTEGER aid;
SQLCHAR    s[70];
SQLINTEGER ind;

rc = SQLAllocStmt(hdbc, &hstmt);

rc = SQLPrepare(hstmt, stmt, SQL_NTS);

printf("Enter Author ID:  "); scanf("%ld",&aid);
printf("Enter Author URL: "); scanf("%s", s);

rc = SQLBindParameter(hstmt, 1,
                      SQL_PARAM_INPUT, SQL_C_CHAR,
                      SQL_CHAR, 0, 0, s, 70, &ind);

rc = SQLBindParameter(hstmt, 2,
                      SQL_PARAM_INPUT, SQL_C_SLONG,
                      SQL_INTEGER, 0, 0, &aid, 0, NULL);

rc = SQLExecute(hstmt);
```

Late Binding of Parameters

- ▶ Executing a statement *without* binding all parameters can be accomplished by setting the last argument of `SQLBindParameter` to point to the constant `SQL_DATA_AT_EXEC`.
⇒ results in an `SQL_NEED_DATA` error code
- ▶ Parameter values can be supplied when detecting this error code by repeated use of two functions.
 - `SQLParamData` (to determine which argument)
 - `SQLPutData` (to supply data)
- ▶ Allows updating several rows using one update statement, but supplying different values for each individual row to be updated.

Answers in ODBC

A number of functions exists to obtain output values computed by an SQL statement.

- ▶ Number of affected rows:

`SQLRowCount`

- ▶ Answers to queries:

`SQLBindCol` (to bind variables before execution)

`SQLGetData` (to get values after execution)

- ▶ Obtaining the next tuple of a query evaluation:

`SQLFetch`

⇒ the result of `SQLFetch` is just a result code

A Query with SQLBindCol

```
SQLCHAR  sqlstmt[] = "SELECT pubid, title FROM publication";

SQLINTEGER  rows;
struct { SQLINTEGER ind;
        SQLCHAR    s[70];
        } pubid, title;

rc = SQLAllocStmt(hdbc, &hstmt);

rc = SQLExecDirect(hstmt, sqlstmt, SQL_NTS);

rc = SQLBindCol(hstmt, 1, SQL_C_CHAR,
                (SQLPOINTER)pubid.s, 8, &pubid.ind);
rc = SQLBindCol(hstmt, 2, SQL_C_CHAR,
                (SQLPOINTER)title.s, 70, &title.ind);

while ((rc = SQLFetch(hstmt)) == SQL_SUCCESS)
    printf("%-8.8s %-70.70s\n", pubid.s, title.s);

rc = SQLRowCount(hstmt, &rows);
printf(" %d rows selected\n", rows);

rc = SQLFreeStmt(hstmt, SQL_DROP);
```

A Query with SQLGetData

```
SQLCHAR  sqlstmt[] = "SELECT pubid, title FROM publication";

SQLINTEGER  rows;
struct { SQLINTEGER ind;
         SQLCHAR    s[70];
         } pubid, title;

rc = SQLAllocStmt(hdbc, &hstmt);

rc = SQLExecDirect(hstmt, sqlstmt, SQL_NTS);

while ((rc = SQLFetch(hstmt)) == SQL_SUCCESS) {
    rc = SQLGetData(hstmt, 1, SQL_C_CHAR,
                    (SQLPOINTER) pubid.s, 8, &(pubid.ind));
    rc = SQLGetData(hstmt, 2, SQL_C_CHAR,
                    (SQLPOINTER) title.s, 70, &(title.ind));
    printf("%-8.8s %-70.70s \n", pubid.s, title.s);
}

rc = SQLRowCount(hstmt, &rows);
printf(" %d rows selected\n", rows);

rc = SQLFreeStmt(hstmt, SQL_DROP);
```

Column Descriptions in ODBC

There are functions to determine the number of results columns for queries, and column name and type information for each.

```
SQLNumResultCols(hstmt, &num)
SQLDescribeCol(hstmt, ColNo,
               ColNamebuf, sizeof(ColNamebuf),
               NULL,
               &sqltype, &sqlprec, &sqlscale,
               &ifNullable );
```

Example:

```
SQLINTEGER sqlprec;
SQLSMALLINT i, num, sqltype, sqlscale, nullable;
SQLCHAR name[32];

rc = SQLNumResultCols(hstmt, &num);

for (i=0; i<num; i++) {
    rc = SQLDescribeCol(hstmt, i+1, name, 32, NULL,
                       &sqltype, &sqlprec, &sqlscale, &nullable);
    printf("attribute %d is %s (%d,%ld,%d,%d)\n"
           i, name, sqltype, sqlprec, sqlscale, nullable);
}
```

Transaction Management in ODBC

- ▶ The start of a new transaction happens implicitly with executable SQL commands:

`SQLPrepare,`
`SQLExecute,`
`SQLExecDirect, etc.`

- ▶ A function is called to signal the end of a transaction:

`SQLTransact(henv, hdbc, <what>)`

where

`<what> = SQL_COMMIT, or`
`<what> = SQL_ROLLBACK`

ODBC Summary

- ▶ ODBC is at least as expressive as dynamic embedded SQL.
 - ⇒ not bound to a particular RDBMS engine
 - ⇒ single threads can access multiple RDBMS engines
- ▶ All statements are *dynamic*.
 - ⇒ no pre-compilation
 - ⇒ requires explicit binding of parameters
(user is responsible for ensuring types match)
- ▶ An almost standard (ODBC, X/Open).
 - ⇒ supported by almost all RDBMS engines
 - ⇒ there are has 100's of functions